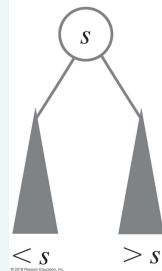


Balanced Search Trees

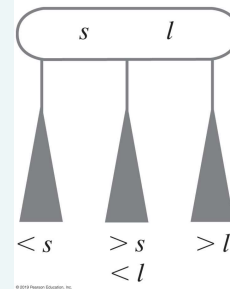
2-3 Trees

- General search tree whose interior nodes must have either two or three children
 - A 2-node contains one data item s and has two children
 - A 3-node contains two data items, s and l , and has three children

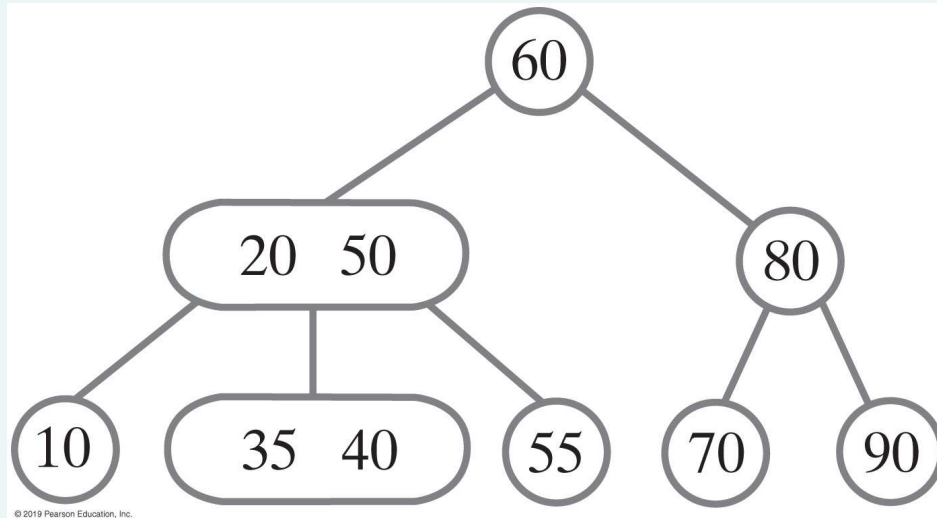
(a) A 2-node



(b) A 3-node



2-3 Trees



Building 2-3 Trees

(a) After adding 60,
the tree is a 2-node



60

(b) After adding 50,
the tree is a 3-node

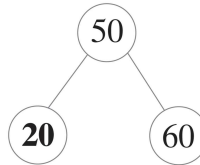


50 60

(c) A 3-node cannot
accommodate 20,
so it must split

20 50 60 → Split

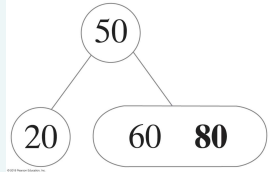
(d) After adding 20



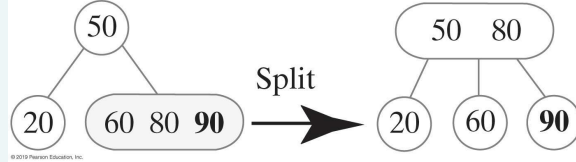
An initially empty 2-3 tree after three additions

Building 2-3 Trees

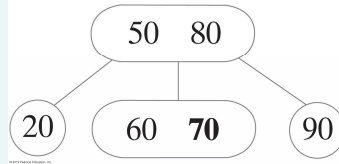
(a) After adding 20



(b) Splitting the leaf and adding 90



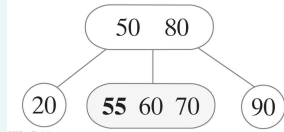
(c) After adding 70



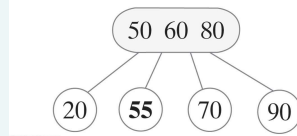
The 2-3 tree after three additions

Building 2-3 Trees

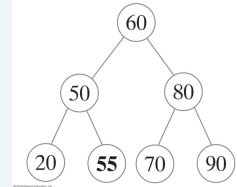
(a) 55 belongs in the middle leaf, but it has no room



(b) The leaf splits, but the root has no room for the 60 that moves up



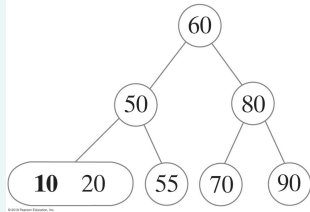
(c) The tree after the root splits



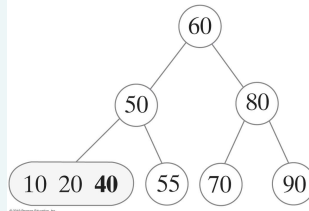
Adding 55 to the 2-3 tree in Figure 28-14c causes a leaf and then the root to split

Building 2-3 Trees

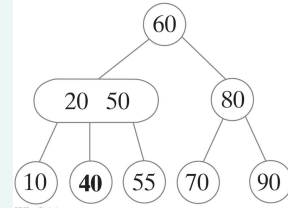
(a) After adding 10



(b) 40 belongs in a leaf that has no room

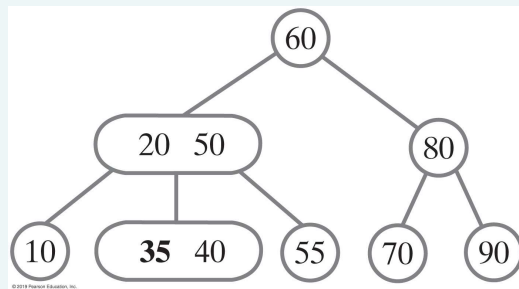


(c) The tree after the leaf splits



Adding 10 and 40 to the 2-3 tree in Figure 28-15c

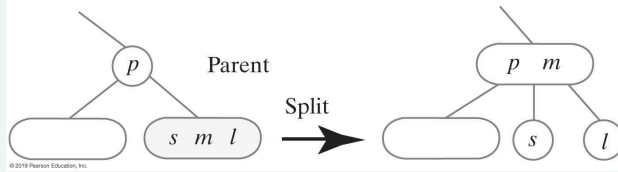
Building 2-3 Trees



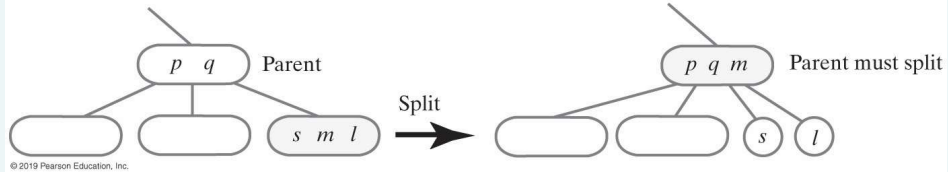
The 2-3 tree in Figure 28-16c after adding 35

Splitting Nodes During Addition

(a) Splitting a leaf when its parent has one entry

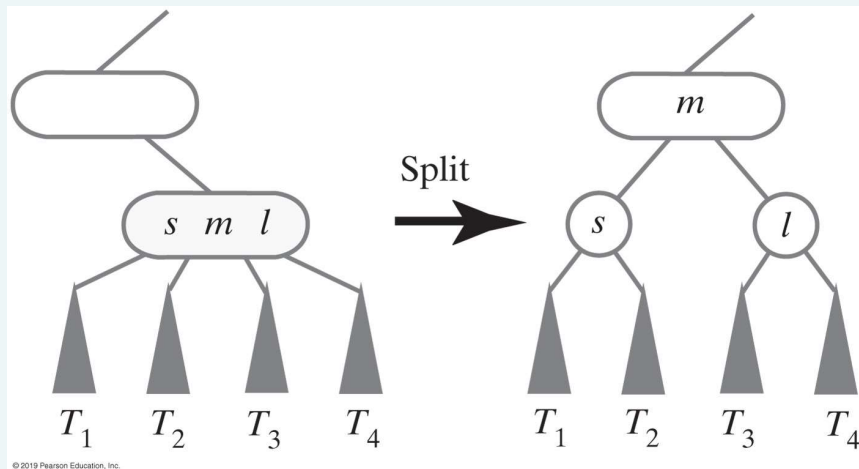


(b) Splitting a leaf when its parent has two entries



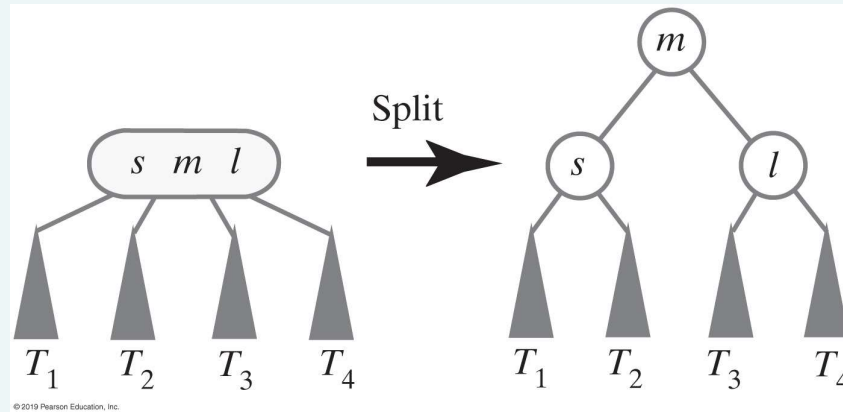
Splitting a leaf to accommodate a new entry

Splitting Nodes During Addition



Splitting an internal node to accommodate a new entry

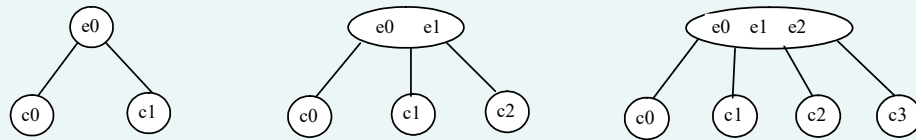
Splitting Nodes During Addition



Splitting the root to accommodate a new entry

What is 2-4 Tree?

A *2-4 tree*, also known as a *2-3-4 tree*, is a *complete balanced* search tree with all leaf nodes appearing on the same level. In a 2-4 tree, a node may have one, two, or three elements. An interior *2-node* contains one element and two children. An interior *3-node* contains two elements and three children. An interior *4-node* contains three elements and four children.



12

Sometimes called a 2-3-4 tree

General search tree

Interior nodes must have either two,
three, or four children

Leaves occur on the same level

This tree also contains 4-nodes.

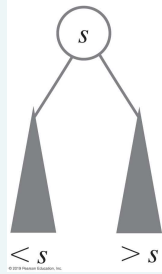
A 4-node contains three data items s , m ,
and l and has four children.

A 2-4 tree tends to be shorter than a corresponding binary search tree, since a 2-4 tree node may contain two or three elements.

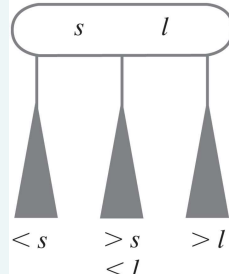
2-4 Trees

- Rules for placing data items in the nodes of a 2-4 tree
 - A 2-node must contain a single data item whose search keys satisfy the relationships pictured in Fig. a
 - A 3-node must contain two data items whose search keys satisfy the relationships pictured in Fig. b
 - A 4-node must contain three data items whose search keys S, M, and L satisfy the relationship pictured in Fig. c
 - A leaf may contain either one, two, or three data items

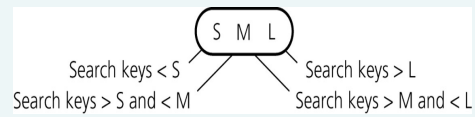
(a) A 2-node



(b) A 3-node



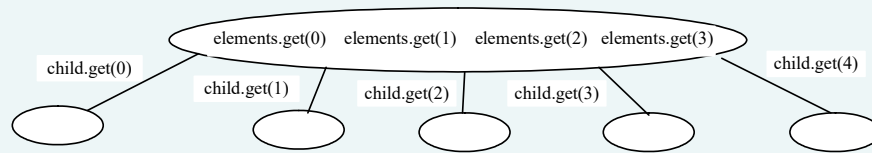
(c) A 4-node



13 A-13

Searching an Element

Searching an element in a 2-4 tree is similar to searching an element in a binary tree. The difference is that you have to also search an element within a node in addition to searching elements along the path. To search an element in a 2-4 tree, you start from the root and scan down. If an element is not in the node, move to an appropriate subtree. Repeat the process until a match is found or you arrive at an empty subtree.



Inserting an Element

To insert an element to a 2-4 tree, locate a leaf node in which the element will be inserted. If the leaf node is a 2-node or 3-node, simply insert the element into the node. If the node is a 4-node, inserting a new element would cause an *overflow*. To resolve overflow, perform a *split* operation.

15

Recall how we add a new entry to a 2-3 tree. We make comparisons along a path that begins at the root and ends at a leaf. At this point, if the leaf is a 3-node, it already contains two data entries, and so we must split it.

Since an entry would now move up a level, this split could require splits in nodes above the leaf. Thus, adding to a 2-3 tree can require us to retrace the path from the leaf back to the root.

In a 2-4 tree, we avoid this retrace by splitting each 4-node **as soon as we first consider it during the search from root to leaf**.

After a split, the next node along the comparison path is the result of the split, and so is not a 4-node.

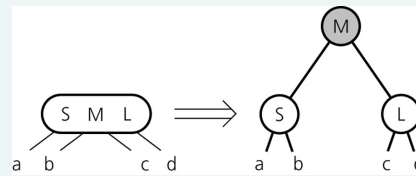
If this node has a 4-node child that we consider next, it has room for the entry that moves up from this child.

No other splits occur, as would happen in a 2-3 tree.

2-3-4 Trees: Splitting 4-nodes During Insertion

- The 4-node that is split will
 - Be the root, or
 - Have a 2-node parent, or
 - Have a 3-node parent
 - Have a 4-node parent

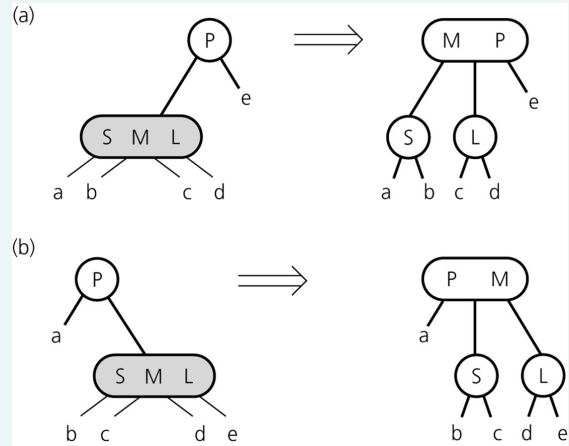
Splitting a 4-node root during insertion



13 A-16

2-3-4 Trees: Splitting 4-nodes During Insertion

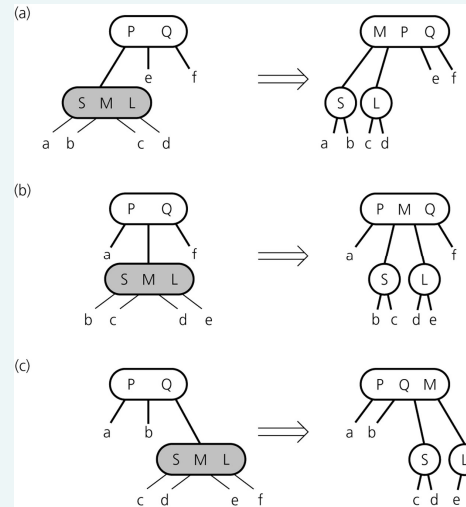
Splitting a 4-node whose parent is a 2-node during insertion



13 A-17

2-3-4 Trees: Splitting 4-nodes During Insertion

Splitting a 4-node whose parent is a 3-node during insertion



13 A-18

Adding 60 to an initially empty 2-4 tree

(a) After adding 60



© 2019 Pearson Education, Inc.

Adding 50 to 2-4 tree

(b) After adding 50



© 2019 Pearson Education, Inc.

Adding 20 to 2-4 tree

(c) After adding 20



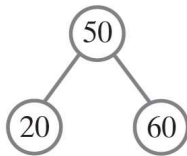
© 2019 Pearson Education, Inc.

Adding 80 and 90 to the tree

(c) After adding 20

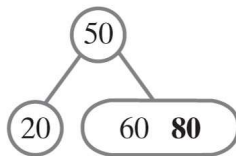
20 50 60

(a) After splitting the 4-node

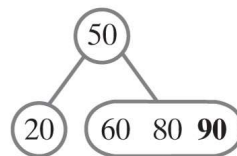


© 2019 Pearson Education, Inc.

(b) After adding 80

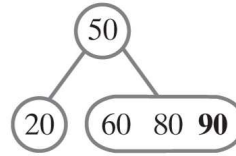


(c) After adding 90

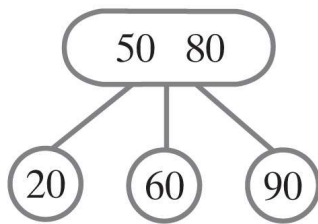


Adding 70 to the 2-4 tree

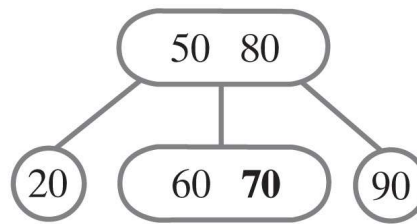
(c) After adding 90



(a) After splitting the 4-node leaf



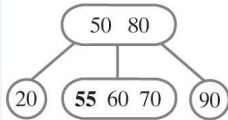
(b) After adding 70



© 2019 Pearson Education, Inc.

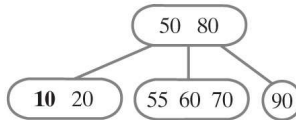
Adding 55, 10, and 40 to the 2-4 tree

(a) After adding 55

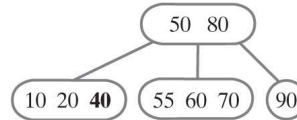


© 2019 Pearson Education, Inc.

(b) After adding 10

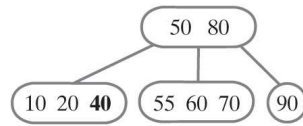


(c) After adding 40

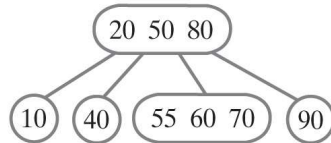


Adding 35 to the 2-4 tree

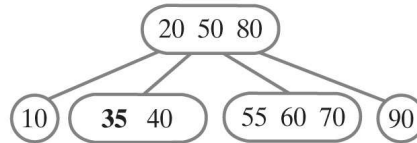
(c) After adding 40



(a) After splitting the 4-node leaf encountered while searching for a place for 35



(b) After adding 35



© 2019 Pearson Education, Inc.

Deleting an Element

To delete an element from a 2-4 tree, first search the element in the tree to locate the node that contains the element. If the element is not in the tree, the method returns false. Let **u** be the node that contains the element and **parentOfu** be the parent of **u**. Consider three cases:

Case 1: **u** is a leaf 3-node or 4-node. Delete **e** from **u**.

Case 2: **u** is a leaf 2-node. Delete **e** from **u**. Now **u** is empty. This situation is known as *underflow*. Perform appropriate operations to remedy an underflow.

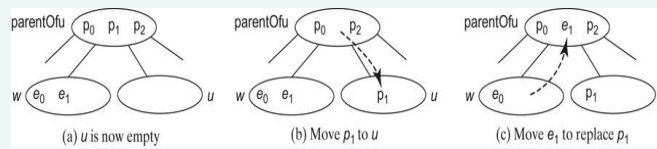
Case 3: **u** is a non-leaf node.

Transform each 2-node encountered into a 3-node or a 4-node

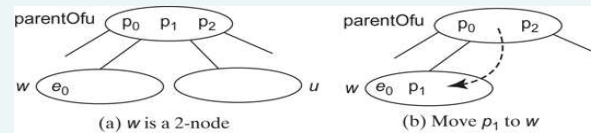
26

Deleting an Element

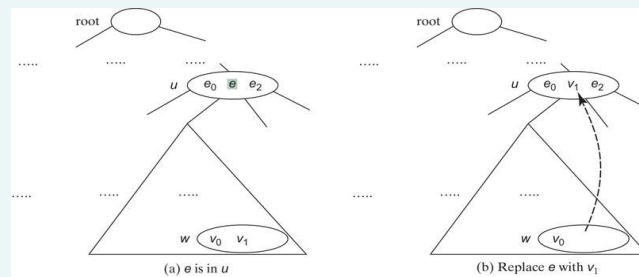
- Case 2: **u** is a leaf 2-node. Delete **e** from **u**. Now **u** is empty. This situation is known as *underflow*.
- Consider to subcases:
 - Case 2.1: **u**'s immediate left or right is a 3- or 4-node. Let the node be **w**, (assume **w** is a left sibling of **u**)



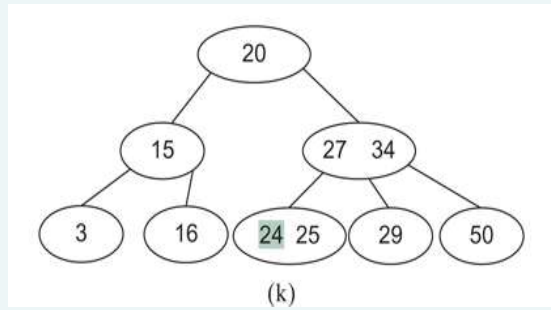
- **Case 2.2:** Both u 's immediate left and right siblings are 2-node if they exist (u may have only one sibling). Let the node be w , (assume w is a left sibling of u). Perform a **fusion** operation that discards u and moves an element from $parentOfu$ to w . If $parentOfu$ becomes empty, repeat Case 2 recursively to perform a transfer or a fusion on $parentOfu$.

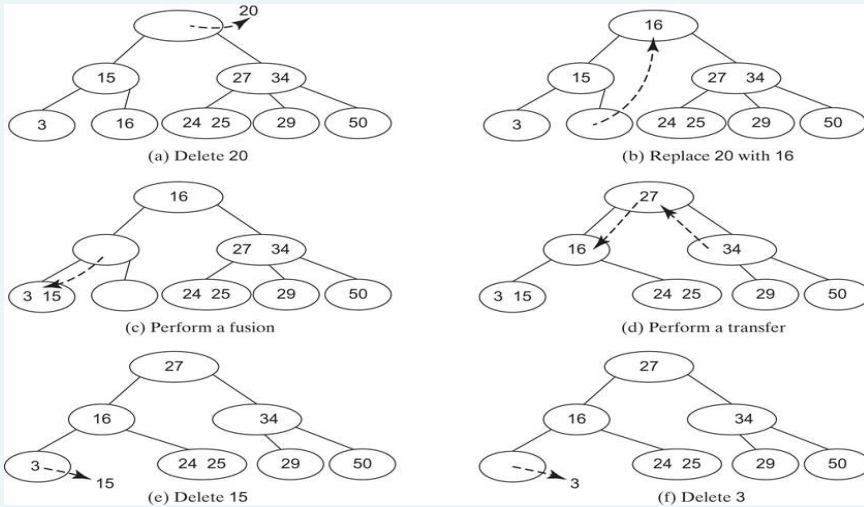


Case 3: u is a nonleaf node. Find the rightmost leaf node in the left subtree of e . Let this node be w . Move the last element in w to replace e in u . If w becomes empty, apply a transfer or fusion operation on w .



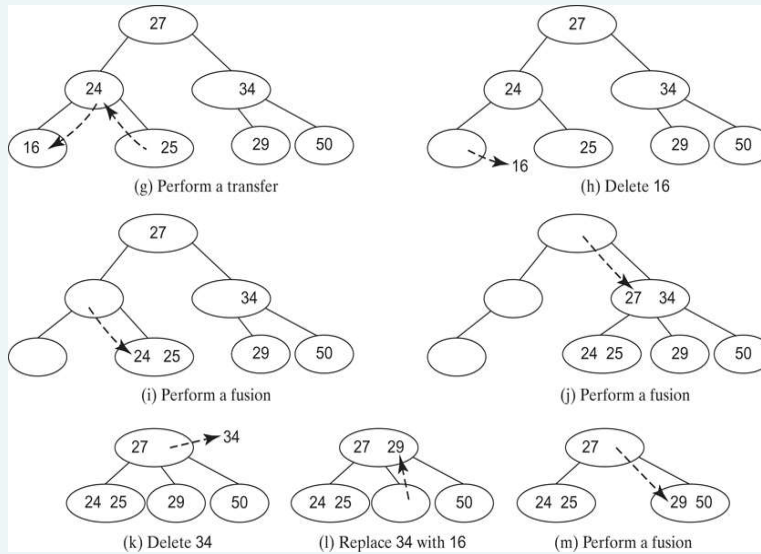
Deleting 20





© 2011 Pearson Addison-Wesley. All rights reserved

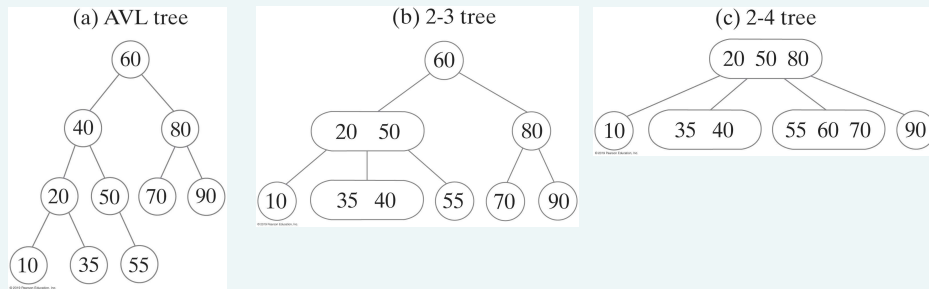
13 A-31



2-3-4 Trees: Concluding Remarks

- Advantage of 2-3 and 2-3-4 trees
 - Easy-to-maintain balance
- Insertion and deletion algorithms for a 2-3-4 tree require fewer steps than those for a 2-3 tree
- Allowing nodes with more than four children is counterproductive

Building 2-4 Trees - A Comparison



- Three balanced search trees obtained by adding 60, 50, 20, 80, 90, 70, 55, 10, 40, and 35

Figure 28-28 compares the AVL tree, 2-3 tree, and the 2-4 tree.

The AVL tree is a balanced binary search tree of height 4. The other trees are completely balanced general search trees. The height of the 2-3 tree is 3; the height of the 2-4 tree is 2.

In general, 2-4 trees are shorter than 2-3 trees, which are shorter than AVL trees.

Searching a balanced binary search tree, such as an AVL tree, is an $O(\log n)$ operation. Since 2-3 and 2-4 trees are no taller than a corresponding AVL tree, we usually can search them by examining fewer nodes.

However, 3-nodes and 4-nodes contain more entries than 2-nodes, and so they require a **longer search time**.

In general, searching an AVL, 2-3, or 2-4 tree is an $O(\log n)$ operation.

A 2-3 tree is appealing because maintaining its balance is easier than for an AVL tree.

Maintaining the balance of a 2-4 tree is even easier. But defining search trees whose nodes contain more than three data items is usually counterproductive, because the number of comparisons per node increases. Such a search tree is attractive when it is maintained in external storage, such as a disk, instead of internal memory.